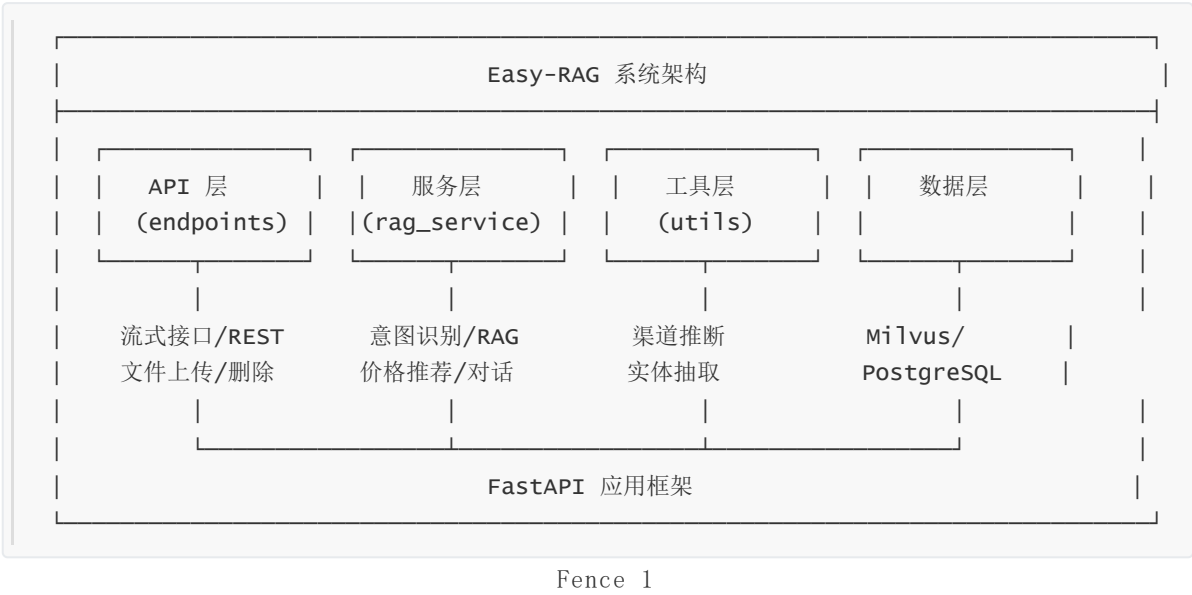


广诚智能问答项目功能逻辑与执行流程文档

1. 目录

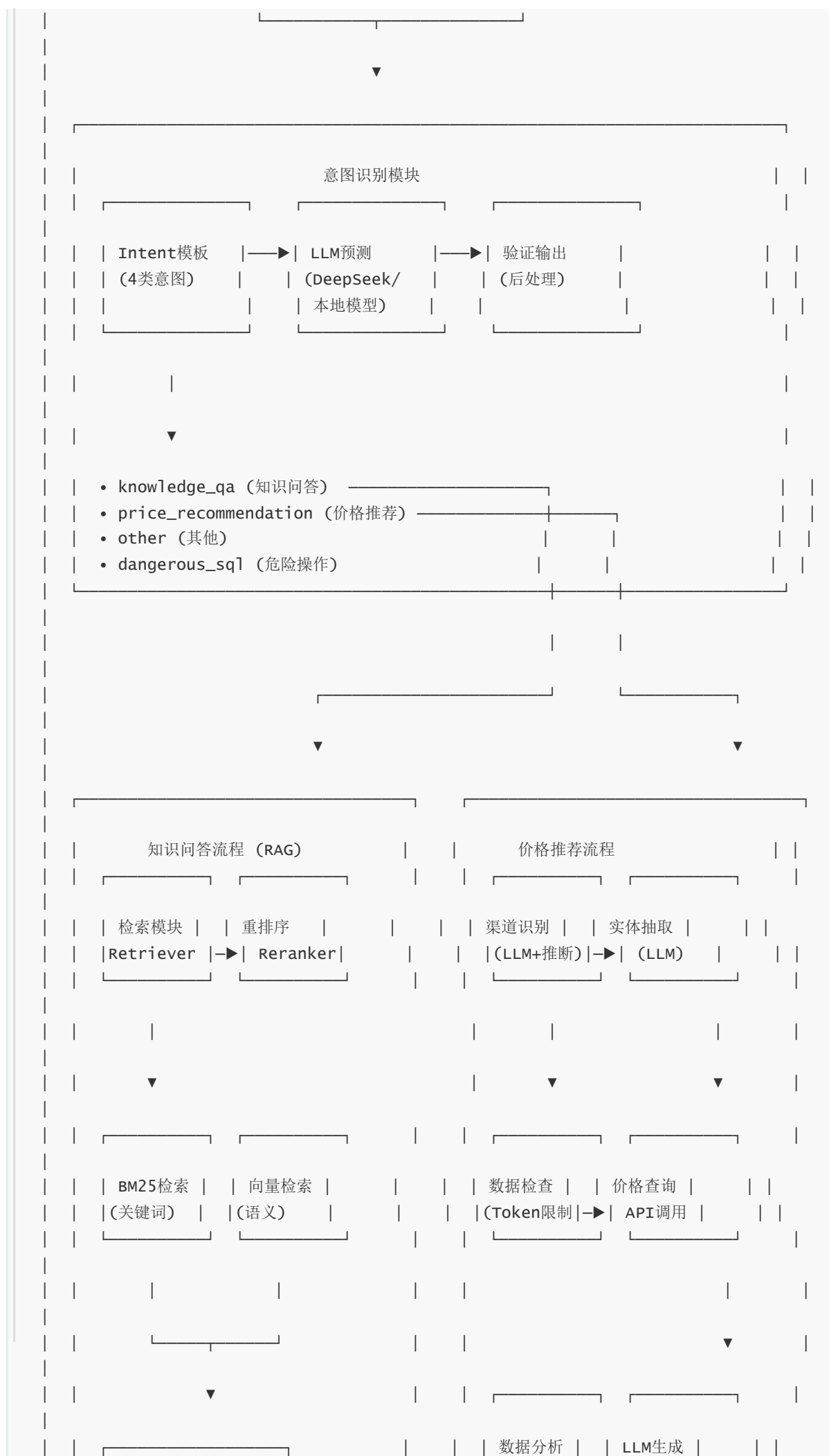
- 1. [系统整体架构](#)
- 2. [核心功能模块关系](#)
- 3. [API 执行流程详解](#)
 - 意图识别接口
 - 知识问答流式接口
 - 价格推荐流式接口
 - 直接价格查询接口
 - 文件管理接口
- 4. [对话管理与渐进式实体补全](#)
- 5. [数据流转全景](#)
- 6. [关键工具类说明](#)
- 7. [项目结构一览](#)

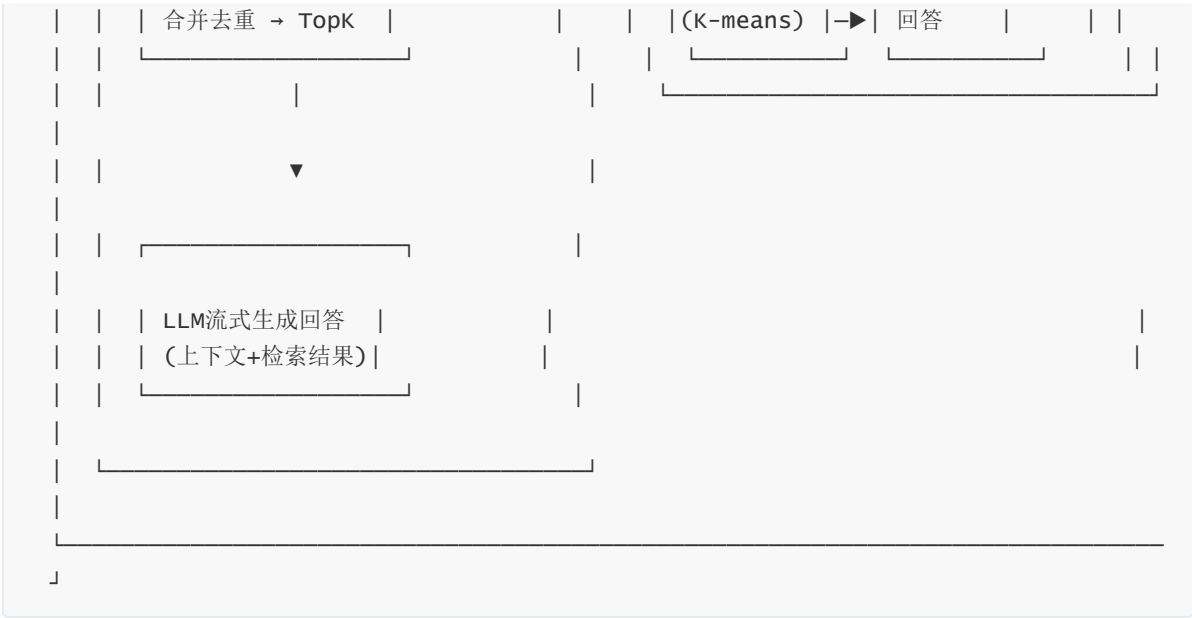
2. 系统整体架构



3. 核心功能模块关系



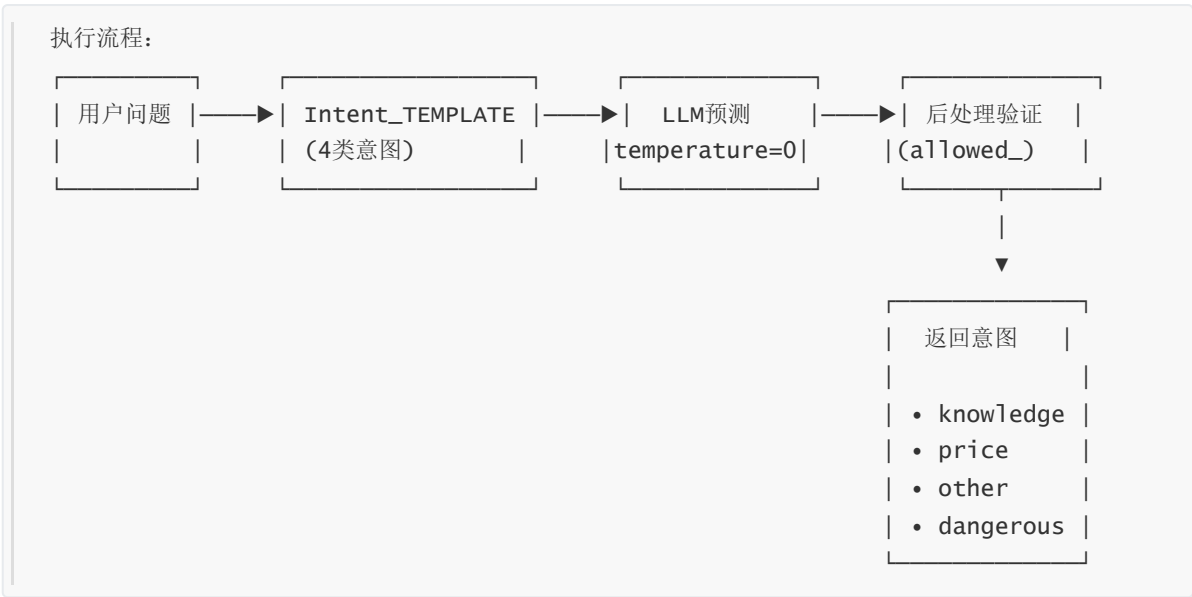




Fence 2

4. API 执行流程详解

4.1 1. 意图识别接口 (POST /api/v1/query/intent)



Fence 3

支持的意图类别:

意图	说明	路由目标
knowledge_qa	知识问答	/api/v1/query/stream
price_recommendation	价格推荐	/api/v1/query/price
other	其他问题	通用回答
dangerous_sql	危险操作	拒绝处理

Table 1

4.2 2. 知识问答流式接口 (POST /api/v1/query/stream)



Fence 4

响应格式:

text/plain 流式响应

- └─ token1\n
- └─ token2\n
- └─ ...
- └─ [END]\n ← 生成结束标记
- └─ {metadata} ← JSON 元数据（完整文本 + 上下文）

4.3 3. 价格推荐流式接口 (POST /api/v1/query/price)



• materialName (材料名称) - 必填	
• materialModelSpec (规格型号) - 可选	
• brand (品牌) - 可选	
• province (省份) - 可选	
• city (城市) - 可选	
• startReleaseDate/endReleaseDate (时间范围) - 可选	
Phase 3: 数据量预检查 (_quick_check_data_count)	
• 调用价格 API, 设置 returnTotalCount=1 (只返回总数, 不返回具体数据)	
• 检查数据量是否超过阈值 (1000条)	
• 预估 Token 数量	
• 如果超限, 返回引导用户细化条件的提示	
Phase 4: 价格数据查询 (_query_price_data)	
根据渠道调用不同 API:	
• INFORMATION_PRICE → _get_infor_material()	
信息价查询 (infor/getInforMaterial)	
• MANUFACTURER_PRICE → _get_factory_material()	
厂商报价查询 (factory/getFactoryMaterial)	
• ZC_PRICE → _get_zhicheng_info_material()	
智诚信息价查询 (zhichengInfo/getZhichengInfoMaterial)	

所有 API 支持：Mock 模式 / 真实 API + 签名验证

Phase 5: 数据过滤 (filter_items)

- 基于材料名称进行模糊匹配
- 计算相似度，过滤低相似度结果
- 阈值：T_keep=0.75, T_drop=0.50, alpha=0.4

Phase 6: 价格数据分析 (analyze_by_unit)

1. 按 unit (单位) 分组
2. 对每组进行统计分析：
 - 去除异常值 (IQR 法: $Q1-1.5*IQR \sim Q3+1.5*IQR$)
 - 计算：总数、有效数、价格区间、均值、中位数
 - K-means 聚类推荐价格 (判断是否分档)

Phase 7: 生成 Markdown 表格 (parse_material_response)

- 根据渠道类型选择不同表头
- 格式化价格数据为 Markdown 表格

Phase 8: LLM 生成回答 (stream_price_predict)

Price_Answer_TEMPLATE → 填充用户问题、解析实体、统计结果

	流式返回生成的自然语言分析文本	

Fence 6

响应格式:

text/plain 流式响应
1. 渠道类型: {"channel": "information_price", "confidence": 0.95, ...}
[END]
2. 解析到实体结果: {"materialName": "...", ...}
[END]
3. LLM 生成的分析文本 (流式 tokens)
[END]
4. 完整元数据 JSON (包含 price_data, metadata, success)

Fence 7

4.4 4. 直接价格查询接口 (POST /api/v1/query/price/direct)

直接价格查询流程

请求体结构:

{

"datatype": "informaterial",

数据类型

"question": "用户问题",

"list": [{

材料列表（目前只取第1个）

"materialName": "...",

"province": "...",

"city": "..."

}]

}

执行流程:

校验列表

非空检查

→

提取实体

(直接传入)

→

价格查询

(复用核心)

→

LLM生成

(可选)

特点:

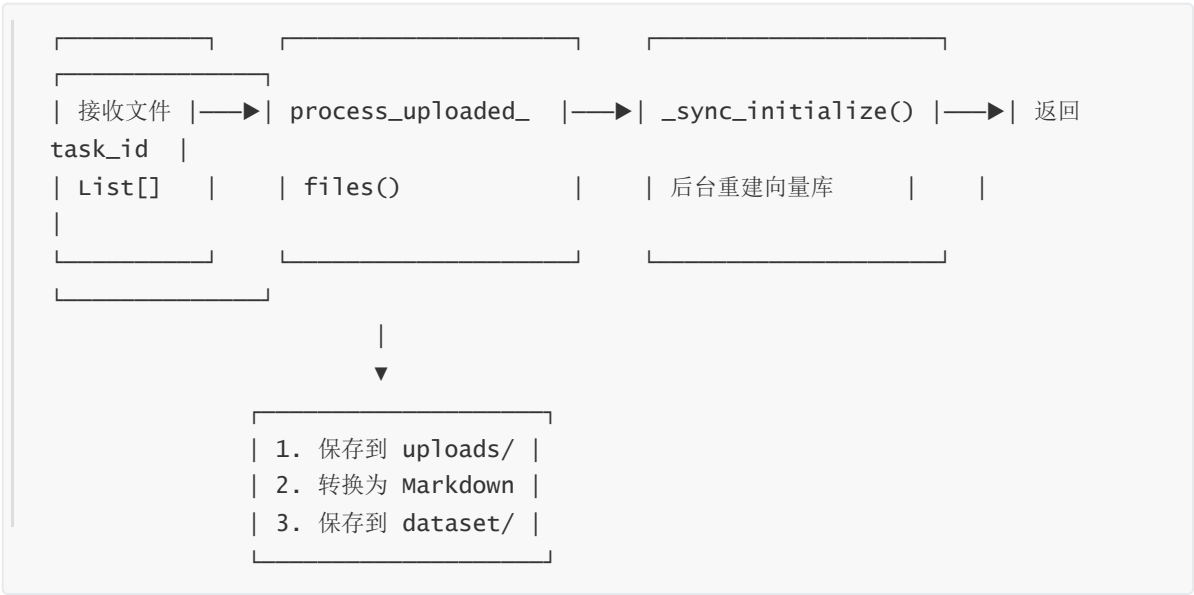
• 用于前端直接传递材料数据场景

• 无需 NLP 解析, 查询更精准

Fence 8

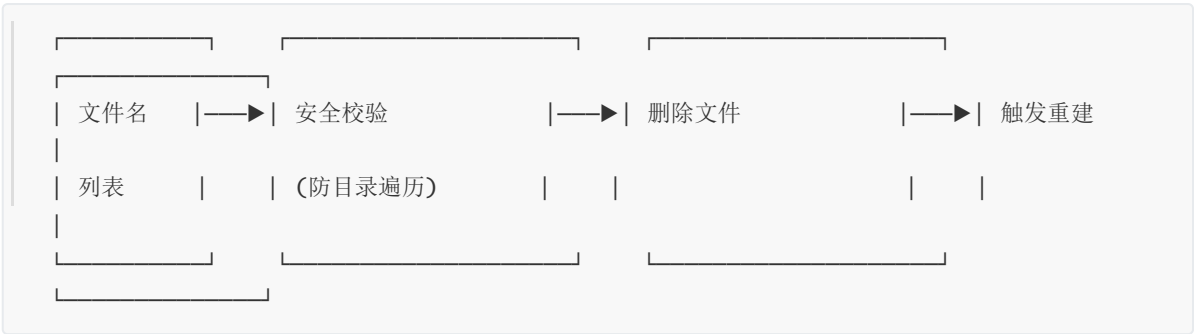
4.5 5. 文件管理接口

4.5.1 上传文件 (POST /api/v1/upload)



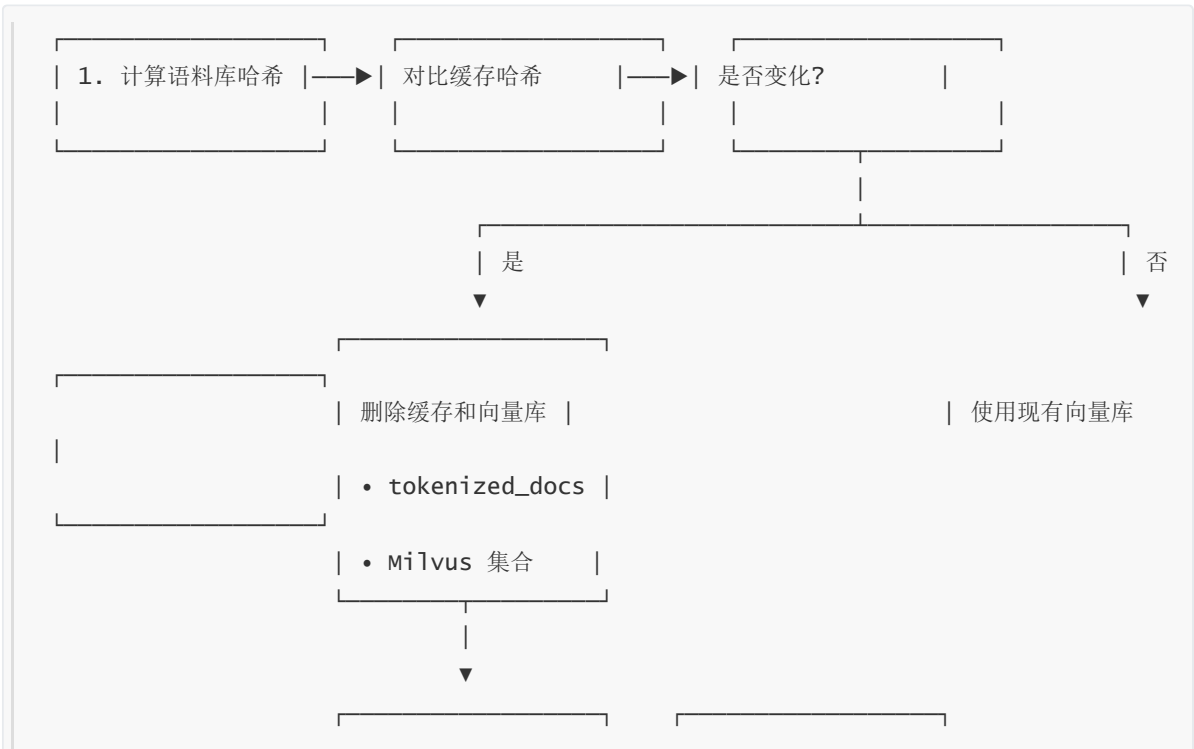
Fence 9

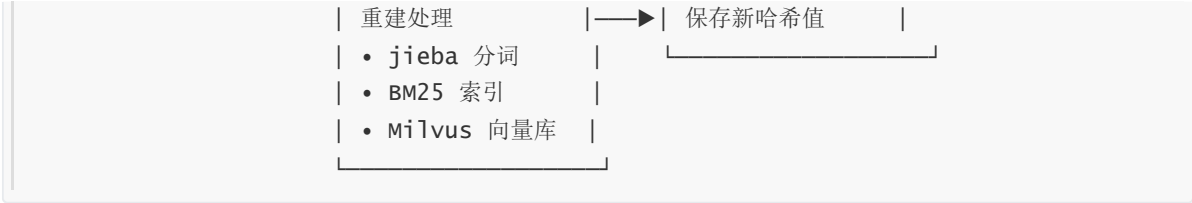
4.5.2 删除文件 (DELETE /api/v1/delete/batch)



Fence 10

4.5.3 知识库重建 (RAGService.initialize)





5. 对话管理与渐进式实体补全

对话式价格查询流程

DialogueManager (对话管理器)

会话状态:

COLLECTING - 收集中

READY - 准备就绪, 可执行查询

COMPLETED - 已完成

EXPIRED - 已过期 (默认10分钟)

关键字段优先级:

优先级	字段	是否必填
1	materialName	✓ 必填
2	province	可选
3	city	可选
4	materialModelSpec	可选
5	brand	可选

渐进式补全流程示例:

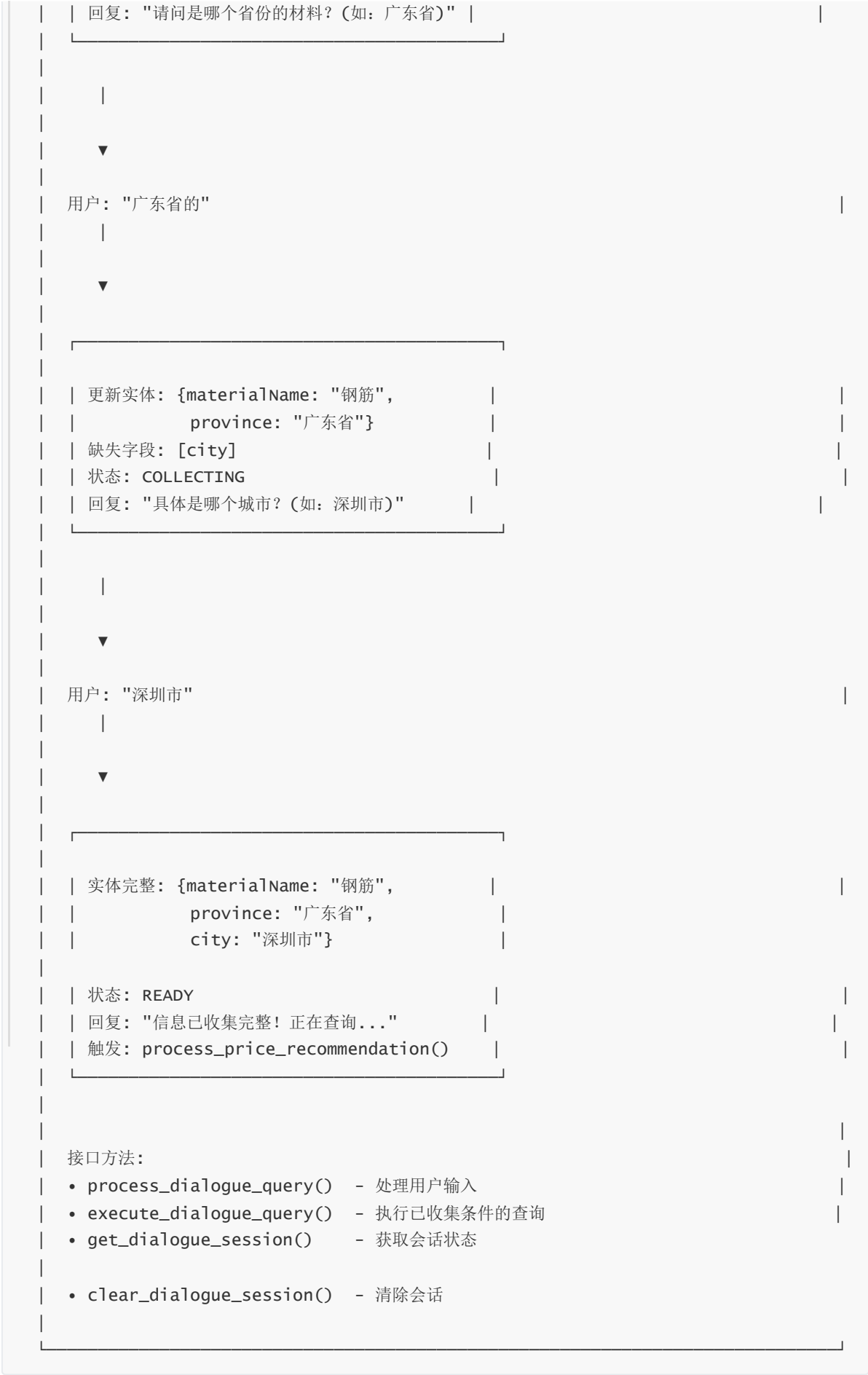
用户: "查一下钢筋价格"

▼

识别实体: {materialName: "钢筋"}

缺失字段: [province, city]

状态: COLLECTING



6. 数据流转全景

数据流转全景图

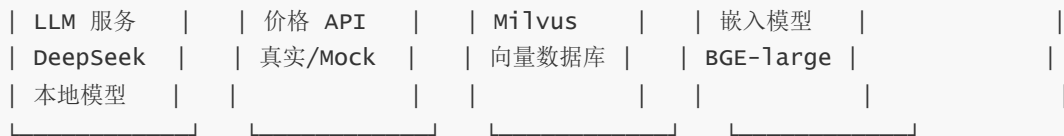
用户输入



存储层



外部依赖





Fence 13

7. 关键工具类说明

工具类	文件路径	核心功能	关键方法
<code>ChannelInferencer</code>	<code>app/utls/channel_inferencer.py</code>	智能渠道推断	<code>infer()</code> - 基于关键词权重和实体特征推断渠道
<code>DialogueManager</code>	<code>app/utls/dialogue_manager.py</code>	对话状态管理	<code>process_user_input()</code> , <code>get_or_create_session()</code>
<code>DialogueSession</code>	<code>app/utls/dialogue_manager.py</code>	会话状态维护	<code>update_entities()</code> , <code>get_next_question()</code>
<code>TokenEstimator</code>	<code>app/utls/data_filter_helper.py</code>	Token 预估	<code>estimate_tokens()</code> - 预估数据 Token 数量
<code>filter_items</code>	<code>app/utls/fillter_tools.py</code>	数据过滤	模糊匹配 + 相似度阈值过滤
<code>analyze_by_unit</code>	<code>app/utls/price_tools.py</code>	价格分析	分组统计 + K-means 聚类 + 异常值处理
<code>recommend_price_mode</code>	<code>app/utls/price_tools.py</code>	价格模式推荐	判断返回单一价格还是分档价格
<code>remove_outliers</code>	<code>app/utls/price_tools.py</code>	异常值处理	IQR 法去除异常价格
<code>generate_signature</code>	<code>app/utls/generate_signature.py</code>	API 签名	请求签名生成与验证
<code>process_uploaded_files</code>	<code>app/services/file_processor.py</code>	文件处理	PDF/Word 转 Markdown

Table 2

8. 项目结构一览



```

|   |— core/
|   |   |— logger.py          # 日志配置
|   |— schemas/              # Pydantic 数据模型
|   |   |— files.py          # 文件相关模型
|   |   |— material.py       # 材料数据模型
|   |   |— price.py          # 价格查询模型
|   |   |— rag.py            # RAG 请求/响应模型
|   |— services/             # 业务逻辑层
|   |   |— file_processor.py # 文件处理服务
|   |   |— rag_service.py    # RAG 核心服务
|   |— utils/                # 工具函数
|   |   |— channel_inferencer.py # 渠道推断器
|   |   |— data_filter_helper.py # 数据量检查
|   |   |— dialogue_manager.py # 对话管理器
|   |   |— fillter_tools.py   # 数据过滤
|   |   |— generate_signature.py # API 签名
|   |   |— price_tools.py     # 价格分析工具
|   |   |— prompt_template.py # 提示词模板
|   |— api_server.py         # FastAPI 应用入口
|   |— config.py             # 全局配置
|   |— embedding_config.py   # 嵌入模型配置
|   |— llm_deepseek.py       # LLM 客户端封装
|   |— retriever.py          # 检索模块 (BM25 + 向量)
|   |— reranker.py           # 重排序模块
|— tests/                   # 测试代码
|— schema.sql               # PostgreSQL 数据库初始化脚本
|— requirements.txt         # Python 依赖
|— docker-compose.yml       # Milvus 服务编排

```

Fence 14

9. 总结

执行流程可以概括为：

1. **入口统一**：所有查询通过 `/api/v1/query/intent` 进行意图识别
2. **分支处理**：
 - **知识问答**：BM25 + 向量检索 → 重排序 → LLM 生成
 - **价格推荐**：渠道识别 → 实体抽取 → 数据查询 → 分析 → LLM 生成
3. **流式响应**：所有接口支持 SSE 流式返回，提升用户体验
4. **对话管理**：支持多轮对话和渐进式实体补全
5. **数据一致性**：语料库变化自动检测，触发向量库重建

整个系统采用**模块化设计**，各组件职责清晰，便于维护和扩展。